KHOJ — Demo Flow Options

What we have built, four ways we could demo it, and what the team needs to decide.

KHOJ is a leaderless drone swarm that finds people cameras cannot see. Several agents share phone signal-strength readings over their own radio mesh and cooperatively locate a transmitter none of them can see. Any agent can die at any moment and the rest carry on — there is no leader and no central computer.

This document is self-contained. If a term is unfamiliar, the glossary on the last page defines every one used here.

1. Where the project actually stands

All of the following is running on **five real ESP32 boards**, measured on the bench — not simulated, not estimated.

0.0%	< 2 s	0.3	4–5 / 5
mesh packet loss across 5 boards	to detect a dead board, independently on each	grid cells — RF localization accuracy achieved	survivors found in sim vs 3–4 for the baseline

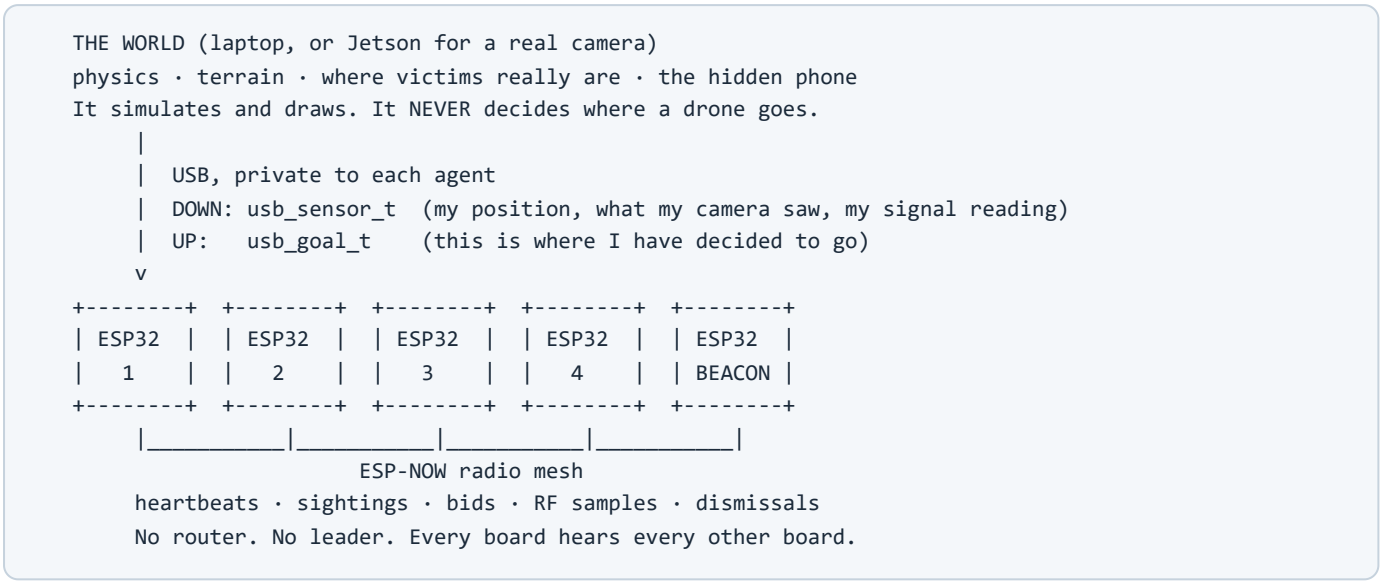
Capability	Status	What it means
ESP-NOW mesh	WORKING	5 boards broadcasting 23-byte packets at 5 Hz. No router, no pairing, no internet.
Failure detection	WORKING	2 s of silence and every surviving board independently marks that board dead. Nobody announces it.
Shared positions	WORKING	Each board takes its position over USB and shares it on the mesh, so every board knows where every drone is.
Cooperative RF localization	WORKING	Agents share signal readings and converge on a hidden transmitter to 0.3 cells.
Belief map, auction, re-observation, Hive-Mind	PYTHON ONLY	**Exists, but only on the laptop.** Not yet running on the boards. This is the gap the demo choice hinges on.

The one gap that matters

The boards cannot yet handle a sighting. If a camera says “I think that is a person, 41% confident”, the boards have no logic for it. Deciding who investigates a sighting currently happens in Python on the laptop.

Path A = port that logic to the boards (~250 lines of C: a task list, announcing, bidding, and the shared winner rule). **Path B** = leave it on the laptop. Both are honest; Path A makes the “the drones decide, not the laptop” claim literally true. **We are attempting Path A.**

2. How the system fits together



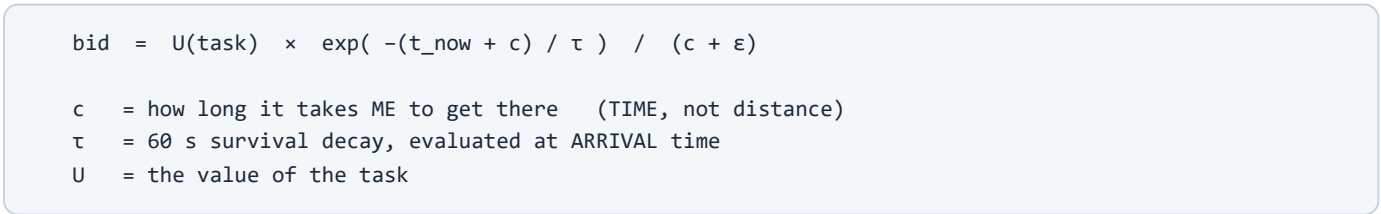
The key idea: the laptop owns *reality*; the boards own the *decisions*. Each board holds its own private belief about where survivors are, built only from what it personally sensed plus what peers broadcast. No board can read another's mind. That separation is the whole reason this is a swarm and not a puppet show.

How five boards agree without a leader

1. A board sees something uncertain and **broadcasts** it as a task.
2. Every board computes **its own bid**: how much value can I deliver per second if I go?
3. Every board **broadcasts its bid**. Now all boards hold identical data.
4. Every board runs the **same rule** — highest bid wins, ties go to the lowest ID.

Same data plus same rule equals the same winner, computed five times independently. **Nobody announces the result and there is no referee to kill.** That is why pulling a board's power changes nothing about how the rest decide.

The bid — one formula for every kind of task



Dividing by time makes the bid **expected survivors per second**, so the swarm always takes the most useful action per second. Evaluating the decay at arrival means a survivor you would reach late is worth less — it prices **urgency**, not just distance.

3. The four demo options

All four use the same boards and the same firmware. They differ in what the judge sees and how much new work each needs.

Option 1 — Jetson as one agent's real eyes

What the judge sees

A camera watches a scene. A person is partly hidden. YOLO returns a real, unconvincing number — 0.41. The swarm decides that is worth a second look, picks who goes, the camera moves to a new angle, and the two independent looks combine into a confident CONFIRMED.

Physical setup

Jetson + camera on a tripod. One ESP32 plugged into the Jetson. The other four boards on the laptop. All five on the mesh.

Exact flow, per frame

1. Jetson captures a frame; YOLO returns `(class, box, confidence)`.
2. Bridge script picks the best `person` box → confidence 0.41.
3. Bridge converts the box centre from pixels to a grid cell, e.g. (12.4, 7.1).
4. Bridge sends `usb_sensor_t` to its ESP32 with `has_detection=1, det_conf=41`.
5. That board sees 41 is in the uncertain band (15–80) and **broadcasts a REOBSERVE task**.
6. All boards bid; all boards independently pick the same winner.
7. Winner replies `usb_goal_t` with the task position and state REOBSERVE.
8. It approaches from a **different angle** and looks again → 0.63.
9. The two looks fuse mathematically → 0.87 → **CONFIRMED**.
10. Had it fused below 0.15 → **DISMISSED**, broadcast to all boards, never re-checked.

Needs: the Jetson bridge (new) + Path A. **Weakness:** only one agent has real perception; the other four are simulated.

Option 2 — The tabletop disaster site RECOMMENDED

What the judge sees

A table laid out as a miniature disaster site — rubble props, a jacket, a hot laptop, a doll. A camera looks straight down at it. On screen, five drones sweep across that real scene and pick out what is on the table.

The idea

The table **is** the 32×32 search grid. Each agent's position maps to a rectangle of the camera image. When agent 3 is at cell (12, 7), the Jetson **crops that rectangle** and runs YOLO on just that crop. **All five agents get real detections, from their own footprints, out of a single camera.**

Exact flow, per frame

1. One frame captured of the whole table.
2. For each agent: take its current position, convert to a pixel rectangle, crop.
3. Run YOLO on each crop → a separate confidence per agent.
4. Send **each agent its own private** `usb_sensor_t` with its own detection.
5. From here identical to Option 1, steps 5–10.

Why this is the pick: every agent really perceives, the swarm sweeps a physical scene, and “each agent sees only its own footprint” becomes literally visible. A judge can move an object mid-demo and watch the swarm react.
Needs: Option 1’s bridge plus a crop loop and one calibration number (pixels per cell).

Option 3 — The two-victim tableau

What the judge sees

Two victims on the table. **Victim A** is partly visible under a cloth — the camera can just about find it. **Victim B** is sealed inside a cardboard box — **no camera can ever see it** — with the beacon board inside.

The swarm finds A with the camera and B by radio. Then the judge lifts the box.

Exact flow

1. Swarm sweeps the table exactly as in Option 2.
2. Victim A → 0.41 → second look → 0.87 → **CONFIRMED by camera**.
3. Meanwhile agents share `RF_SAMPLE` readings and hill-climb toward the box.
4. They converge on it → **CONFIRMED by radio, never seen**.
5. The hot laptop reads 0.34, gets dismissed, and no agent re-checks it.
6. **Judge lifts the box.** The beacon is inside.

The line to say: *"A camera-only swarm finds one of these two. Ours finds both — and the one it cannot see is the one that was buried."* This is the entire pitch as a physical object. **Needs:** Option 2 plus a box. Do both.

Option 4 — The threshold trap, live

What the judge sees

Two counters on the dashboard, updating live during the run:

```
SINGLE-PASS  (what a normal system reports):  4 alarms → 1 real, 3 false
KHOJ         (with re-observation):            1 confirmed, 3 dismissed
```

Same detections, same run. The only difference is whether marginal sightings get a second look. It turns our strongest argument — **a false positive costs a rescue team three minutes, a false negative costs a life** — into a number on screen instead of a claim. **Needs:** ~30 minutes of dashboard work. It is only counting.

4. Recommendation

Build Options 2 + 3 + 4 as one setup. A single tabletop scene, a camera above it, a hidden beacon in a box, and live counters. That one arrangement delivers camera detection, re-observation, Hive-Mind dismissal, the RF hero moment, and the false-alarm argument — all on real hardware a judge can reach out and touch.

Why not fly a drone

A drone flying a pattern shows **motion**. The tabletop shows **the swarm thinking**, which is the thing no other team will have. Building an autonomous flight in the time we have is the classic scope trap, and one real drone beside

> four simulated ones reads as a prop rather than a swarm. If someone can fly it **independently**, record 30 seconds of outdoor footage for the pitch video — it never touches the live demo.

Three beats, all participatory

1. **The judge hides the beacon** → four agents converge on something invisible.
2. **The judge pulls a board's power** → the others notice in 2 s and absorb its work, with nobody in charge.
3. **The judge stands in front of the camera, partly hidden** → 0.41 → second angle → CONFIRMED.

People remember what they *did*, not what they watched.

5. What the team needs to decide

Decision	Options
Which demo flow	Recommendation: Options 2 + 3 + 4 combined as one tabletop setup.
Who builds the physical scene	Table, props, camera mount, and calibrating pixels→grid cells.
Who sets up the Jetson	OS, camera, YOLO printing live detections. Blocks the bridge.
Drone: yes or no	Only as pre-recorded footage, and only if someone is free. Never live.
Feature freeze time	Proposed: stop adding features with 3 hours left. Rehearse ≥5 full runs and record a backup video.

6. Glossary

Term	Meaning
ESP-NOW	Espressif's radio protocol. Boards broadcast directly to each other — no WiFi router, no pairing, no internet.
Agent	One drone's brain — one ESP32 board.
Belief map	A 32×32 grid inside each board holding how likely each cell still hides an unfound victim. **Private to that board.**
Auction	How the swarm splits work. Everyone bids, everyone applies the same rule, everyone gets the same winner — no leader.
Bid	A number meaning "expected survivors per second if I take this task".
REOBSERVE	A task created when a sighting is too uncertain to trust — someone must look again from a different angle.
Log-odds fusion	The maths that combines two independent looks into one confidence. Only fuses views more than 20° apart, so the looks stay independent.
Hive-Mind	When the swarm rules something out, it broadcasts that, and no agent ever re-investigates it. *"The individual forgets; the swarm remembers."*
RSSI	Received signal strength, in dBm. Closer to the transmitter = less negative. −30 is strong, −80 is weak.
RF_SAMPLE	A broadcast saying "at position (x, y) I measured this signal strength". Pooling these is how the swarm locates a hidden phone.
Gradient-seeking	Moving toward a stronger signal rather than calculating distance. Needs no calibration, so it survives concrete and interference.
Beacon	The board playing the victim's phone. It only transmits; it never searches.
Hardware-in-the-loop	Real decision-making hardware driven by a simulated world. Standard practice — PX4 ships a mode for it.
Path A / Path B	Whether the sighting-and-bidding logic runs on the boards (A) or on the laptop (B).

Full technical detail is in `HANDOFF.md` and `README.md` in the repository.